Post-contenido — Manejo del DEBUG: exploración, ensamblado y ejecución paso a paso

Arquitectura de Computadores · Unidad 3 Estudiante: González López **Actividad:** `apellido-post1-u3`

Propósito y estado de la evidencia

El laboratorio se divide en dos partes. La primera usa `R`, `F`, `D`, `E`, `A`, `U` y `T` para observar registros, memoria y codificación de instrucciones; la segunda ensambla y traza sumas, un bucle `LOOP` y su equivalente con `DEC/JNZ`. Los resultados y las capturas incluidos se obtuvieron en una sesión real de DOSBox 0.74-3 con FreeDOS DEBUG, descargado del paquete oficial de FreeDOS DEBUG. En esta ejecución DOS asignó el segmento `0724`; otros equipos pueden mostrar valores diferentes.

Evidencia: las doce imágenes de `capturas/` son capturas directas de la ventana DOSBox durante la ejecución; no se recrearon las salidas. `DEBUG.COM` quedó instalado en `C:\DOSWork\L3P1` y `C:\DOSWork\L3P2` para estas pruebas.

Preparación del entorno

1. Se crea `C:\DOSWork\L3P1` en Windows y se copia allí `DEBUG.COM` del paquete FreeDOS. `L3P1` y `L3P2` se mantienen dentro del límite de 8 caracteres de los nombres DOS.
2. Se inicia DOSBox, se monta la carpeta de trabajo y se entra al laboratorio:

```
Z:\> MOUNT C C:\DOSWork
Z:\> C:
C:\> CD L3P1
C:\L3P1> DEBUG
-
```

1. La Parte 2 se realiza en `C:\DOSWork\L3P2`, con el mismo procedimiento. En esta sesión el segmento fue `0724`; en otra ejecución se debe usar el segmento que muestre DEBUG.

Parte 1 — Exploración, ensamblado y memoria

Checkpoint 1 — Registros con R

Desde el indicador `-`, `R` muestra los registros y las banderas. En la captura real, los valores iniciales incluyen `SP=FFFE`, `IP=0100` y `DS=ES=SS=CS=0724`. `R AX` permite cambiar AX: se escribe `1234` en el indicador de edición y se vuelve a consultar `R AX`; la salida confirma `AX=1234`.

Checkpoint 2 — Relleno y volcado de memoria con F y D

```
-F 200 L 40 AB CD EF
-D 200 23F
```

En FreeDOS DEBUG, `L 40` especifica una longitud de 0x40, es decir, 64 bytes; por eso se separa `L` del valor. El patrón `AB CD EF` se repite desde `DS:0200`. El comando `D 200 23F` muestra el rango equivalente mediante sus direcciones inicial y final. En el volcado, la primera columna es la dirección de inicio de la fila

(`segmento:desplazamiento`), el bloque central muestra los bytes en hexadecimal y la columna derecha representa esos mismos bytes como caracteres. Los bytes que no corresponden a caracteres ASCII imprimibles se muestran como puntos. La captura muestra las cuatro filas del patrón.

La inspección complementaria del PSP se realiza con `D 0 L20`. Sus primeros bytes pueden incluir `CD 20`, la instrucción `INT 20` que DOS coloca en el comienzo del PSP; los demás datos del PSP dependen del entorno y no deben copiarse de un ejemplo como si fueran universales.

Checkpoint 3 — Ensamblado y desensamblado con A y U

Se ingresa el programa en `CS:0100` y se verifica sin ejecutarlo:

```
-A 100
MOV AX,0005
MOV BX,0003
ADD AX,BX
INT 20

-U 100 109
```

La salida medida de FreeDOS DEBUG es:

Dirección	Bytes	Instrucción	Interpretación
`0100`	`B8 05 00`	`MOV AX,0005`	Opcode `B8` y valor inmediato `0005` en little-endian.
`0103`	`BB 03 00`	`MOV BX,0003`	Opcode `BB` y valor inmediato `0003` en little-endian.
`0106`	`01 D8`	`ADD AX,BX`	Suma el contenido de BX a AX.
`0108`	`CD 20`	`INT 20`	Termina el programa DOS.

El programa ocupa 10 bytes. El DEBUG de Microsoft que ilustra la guía puede codificar `ADD AX,BX` como `03 C3`; el ensamblador de FreeDOS usado en esta ejecución escogió `01 D8`, otra codificación válida de la misma operación. `U` muestra los bytes efectivamente escritos en memoria, sin ejecutar el programa.

Checkpoint 4 — Escritura, lectura y direccionamiento directo

```
-F 300 L 10 00
-D 300 30F
-E 300 78 56
-D 300 30F
-A 320
MOV AX,[0300]
INT 20
-U 320 324
-R IP
0320
-T
```

La sesión muestra dieciséis ceros antes de `E`; después, los dos primeros bytes son `78 56` y los catorce restantes continúan en cero. Como x86 almacena palabras en little-endian, esos bytes forman `5678h`. `U` muestra `A1 00 03` para `MOV AX,[0300]`; al ejecutar esa instrucción con `T`, AX pasa a `5678h`. La lectura usa DS por defecto y no debe confundirse con `MOV AX,5678`, que cargaría un valor inmediato.

Decisión técnica 1 — Verificación no destructiva de la memoria

Para comprobar la escritura realizada con `E 300 78 56`, la decisión adecuada es usar `D 300 30F`, porque presenta el contenido del rango sin reemplazarlo. `E` escribe bytes: invocarlo sin una lista de valores abre la

edición interactiva y deja la posibilidad de sobrescribir una ubicación al responder a los valores mostrados. `F` tampoco sirve para verificar, pues rellena un rango con un patrón y, por tanto, altera precisamente el estado que se desea inspeccionar. `R` muestra registros cuando se consulta sin seleccionar uno, pero no muestra el bloque de memoria; además, las variantes de `R` que seleccionan un registro permiten modificarlo. `D` es la opción de solo lectura entre los comandos disponibles para observar ese rango: permite confirmar `78 56` y los ceros restantes sin cambiar la memoria ni el estado de los registros. La captura debe mostrar el volcado posterior a la escritura, no una nueva operación de escritura.

Decisión técnica 2 — Direccionamiento inmediato frente a directo a memoria

`MOV AX,0005` codifica un valor inmediato: sus tres bytes son `B8 05 00`, donde el dato forma parte de la instrucción. `MOV AX,[0300]` también ocupa tres bytes, `A1 00 03`, pero codifica una dirección y requiere leer el operando almacenado en memoria en `DS:0300`. Por ello, la forma directa a memoria requiere un acceso de lectura de datos adicional respecto de obtener el valor inmediato del flujo de instrucciones; el número exacto de ciclos de bus o de reloj depende del procesador y de la implementación, y no se deduce únicamente del tamaño del opcode. Cuando el valor es una constante conocida al ensamblar, la forma inmediata evita esa lectura de datos. Cuando el valor puede cambiar en tiempo de ejecución —como los bytes escritos antes con `E`— la forma directa permite que AX reciba el dato actualizado. `U` verifica ambas codificaciones sin ejecutar ninguna de ellas: muestra `B8 05 00` para la primera y `A1 00 03` para la segunda.

Parte 2 — Trazas y bucles

Las trazas siguientes parten de los valores cargados por el propio programa. En las columnas de banderas se muestran CF, ZF y SF: `–` indica que la instrucción no las modifica, por lo que conservan el estado anterior.

Checkpoint 1 — Programa de suma

El programa se ensambla en `0100`: `MOV AX,000A`, `MOV BX,0005`, `MOV CX,0003`, `ADD AX,BX`, `ADD AX,CX`, `INT 20`. Se verifica con `U 100 10E`, se coloca IP en `0100` con `R IP` y se ejecuta una instrucción por cada comando `T`. FreeDOS DEBUG codificó las dos sumas como `01 D8` y `01 C8`; la guía muestra las formas equivalentes `03 C3` y `03 C1`. En ambos casos el resultado observable es el mismo.

#	Instrucción ejecutada	AX después	BX después	CX después	IP siguiente	CF/ZF/SF
1	`MOV AX,000A`	`000A`	`0000`	`0000`	`0103`	—
2	`MOV BX,0005`	`000A`	`0005`	`0000`	`0106`	—
3	`MOV CX,0003`	`000A`	`0005`	`0003`	`0109`	—
4	`ADD AX,BX`	`000F`	`0005`	`0003`	`010B`	`0/0/0`
5	`ADD AX,CX`	`0012`	`0005`	`0003`	`010D`	`0/0/0`
6	`INT 20`	`0012`	`0005`	`0003`	Terminación	—

El resultado es `10 + 5 + 3 = 18 = 0012h`. Las dos sumas producen un resultado distinto de cero, no generan acarreo fuera de 16 bits y tienen el bit de signo en cero.

Checkpoint 2 — Bucle LOOP

Programa en `0100`: `MOV AX,0000`, `MOV CX,0004`, `ADD AX,0002`, `LOOP 0106`, `INT 20`. FreeDOS DEBUG codifica `ADD AX,0002` como `83 C0 02` y `LOOP` como `E2 FB`: la instrucción LOOP termina en `010B` y el desplazamiento relativo `FBh` es −5, que regresa a `0106`.

#	Instrucción ejecutada	Iteración	AX después	CX después	IP siguiente	¿Salta?	CF/ZF/SF
1	`MOV AX,0000`	—	`0000`	`0000`	`0103`	—	—

#	Instrucción ejecutada	Iteración	AX después	CX después	IP siguiente	¿Salta?	CF/ZF/SF
2	`MOV CX,0004`	—	`0000`	`0004`	`0106`	—	—
3	`ADD AX,0002`	1	`0002`	`0004`	`0109`	—	`0/0/0`
4	`LOOP 0106`	1	`0002`	`0003`	`0106`	Sí	—
5	`ADD AX,0002`	2	`0004`	`0003`	`0109`	—	`0/0/0`
6	`LOOP 0106`	2	`0004`	`0002`	`0106`	Sí	—
7	`ADD AX,0002`	3	`0006`	`0002`	`0109`	—	`0/0/0`
8	`LOOP 0106`	3	`0006`	`0001`	`0106`	Sí	—
9	`ADD AX,0002`	4	`0008`	`0001`	`0109`	—	`0/0/0`
10	`LOOP 0106`	4	`0008`	`0000`	`010B`	No	—
11	`INT 20`	—	`0008`	`0000`	Terminación	—	—

`LOOP` decrementa CX y salta mientras CX no sea cero, sin alterar las banderas. Se ejecutan 11 instrucciones en total al incluir las dos inicializaciones y `INT 20`; el cuerpo del bucle se ejecuta cuatro veces.

Análisis de bytes del bucle LOOP

La guía contiene una inconsistencia: `D CS:100 L0C` solicita 12 bytes, aunque la suma de las longitudes de las instrucciones da 13. Los tamaños son 3 + 3 + 3 + 2 + 2 = 13 bytes, desde `0100` hasta `010C`. Para verlos todos se debe usar `D CS:100 L0D`.

Dirección	Bytes	Instrucción	Longitud
`0100`	`B8 00 00`	`MOV AX,0000`	3
`0103`	`B9 04 00`	`MOV CX,0004`	3
`0106`	`83 C0 02`	`ADD AX,0002`	3
`0109`	`E2 FB`	`LOOP 0106`	2
`010B`	`CD 20`	`INT 20`	2
Total			**13 bytes**

Checkpoint 3 — Bucle equivalente con DEC/JNZ

Programa en `0200`: `MOV AX,0000`, `MOV CX,0004`, `ADD AX,0002`, `DEC CX`, `JNZ 0206`, `INT 20`. FreeDOS DEBUG codifica la suma como `83 C0 02`; `DEC CX` ocupa un byte (`49`) y `JNZ 0206` ocupa dos (`75 FA`), pues el desplazamiento relativo desde `020C` hasta `0206` es −6.

#	Instrucción ejecutada	Iteración	AX después	CX después	IP siguiente	¿JNZ salta?	CF/ZF/SF
1	`MOV AX,0000`	—	`0000`	`0000`	`0203`	—	—
2	`MOV CX,0004`	—	`0000`	`0004`	`0206`	—	—
3	`ADD AX,0002`	1	`0002`	`0004`	`0209`	—	`0/0/0`
4	`DEC CX`	1	`0002`	`0003`	`020A`	—	`0/0/0`
5	`JNZ 0206`	1	`0002`	`0003`	`0206`	Sí	—
6	`ADD AX,0002`	2	`0004`	`0003`	`0209`	—	`0/0/0`
7	`DEC CX`	2	`0004`	`0002`	`020A`	—	`0/0/0`
8	`JNZ 0206`	2	`0004`	`0002`	`0206`	Sí	—
9	`ADD AX,0002`	3	`0006`	`0002`	`0209`	—	`0/0/0`

#	Instrucción ejecutada	Iteración	AX después	CX después	IP siguiente	¿JNZ salta?	CF/ZF/SF
10	`DEC CX`	3	`0006`	`0001`	`020A`	—	`0/0/0`
11	`JNZ 0206`	3	`0006`	`0001`	`0206`	Sí	—
12	`ADD AX,0002`	4	`0008`	`0001`	`0209`	—	`0/0/0`
13	`DEC CX`	4	`0008`	`0000`	`020A`	—	`0/1/0`
14	`JNZ 0206`	4	`0008`	`0000`	`020C`	No	—
15	`INT 20`	—	`0008`	`0000`	Terminación	—	—

Los dos bucles producen `AX=0008` y `CX=0000`. El bucle `LOOP` ejecuta 11 instrucciones; la versión `DEC/JNZ` ejecuta 15: una instrucción adicional de control por cada iteración.

Decisión técnica 3 — Selección de control de bucle

Para este contador simple, la decisión recomendada es `LOOP`: decrementa CX y evalúa si debe saltar en una sola instrucción de dos bytes (`E2 FB`). La alternativa ocupa tres bytes para el control (`DEC CX` con `49` y `JNZ` con `75 FA`) y hace que el procesador ejecute dos instrucciones de control en cada iteración, frente a una. En esta traza de cuatro iteraciones, el total es de 11 instrucciones con `LOOP` y 15 con `DEC/JNZ`, incluyendo inicialización y terminación. `DEC/JNZ` resulta útil cuando la salida debe depender de una comparación más general, por ejemplo `CMP` seguido de una condición como `JE` o `JNE`, o cuando la lógica exige actualizar las banderas mediante una instrucción separada. `U 100` y `U 200` permiten revisar las codificaciones y direcciones sin ejecutar el programa; la diferencia entre las direcciones iniciales y finales permite comparar los tamaños. Por claridad y economía, `LOOP` es la opción apropiada para el caso planteado, no una regla universal para toda clase de bucle.

Decisión técnica 4 — Traza T frente a ejecución G

Para cien iteraciones, `G` es más práctico que emitir `T` repetidamente porque ejecuta las instrucciones hasta el punto de interrupción indicado, por ejemplo `G 20C`, y permite comprobar rápidamente el resultado final. Esa rapidez tiene un costo: no se observan los valores intermedios de AX y CX, ni el instante en que cada rama se toma, por lo que no se puede reconstruir una tabla de traza instrucción por instrucción a partir de ese único resultado. En este laboratorio, `T` es necesario en la suma y en ambos bucles porque cada fila documenta el estado posterior a una instrucción. `G` sirve como verificación final complementaria, no como sustituto de esas trazas. Al detenerse en `020C`, DEBUG queda en la instrucción `INT 20` sin ejecutarla; el estado ya muestra el acumulador final. Después, `R AX` confirma el valor final de AX sin avanzar el programa. Esta combinación distingue una inspección eficiente del resultado de una observación detallada del proceso.

Checkpoint 4 — Repositorio y entrega

La carpeta local contiene este `README.md` y las doce capturas reales listadas en `capturas/README.md`. La guía pide que el repositorio de entrega se llame `apellido-post1-u3` y tenga al menos tres commits descriptivos por cada parte. La actividad está dentro de `TrabajosArqui`, por lo que todavía debe publicarse o copiarse al destino exacto que se comparta con el docente; no se afirma que exista un repositorio independiente con aquel nombre ni se crean commits ficticios. Siguen pendientes los commits requeridos y compartir el enlace en NPLAD UFPS.

Observaciones y conclusión

- Los comandos `F` y `E` escriben memoria; `D` la inspecciona, y `U` traduce bytes a instrucciones sin ejecutarlas.
- `A` escribe instrucciones en memoria. `T` ejecuta una instrucción y permite observar los registros resultantes.
- Los datos inmediatos de 16 bits aparecen en little-endian. Los saltos cortos usan un desplazamiento relativo con signo contado desde la instrucción siguiente.
- `LOOP` decrementa CX sin cambiar las banderas; `DEC` sí actualiza ZF y `JNZ` usa ZF para decidir si salta.

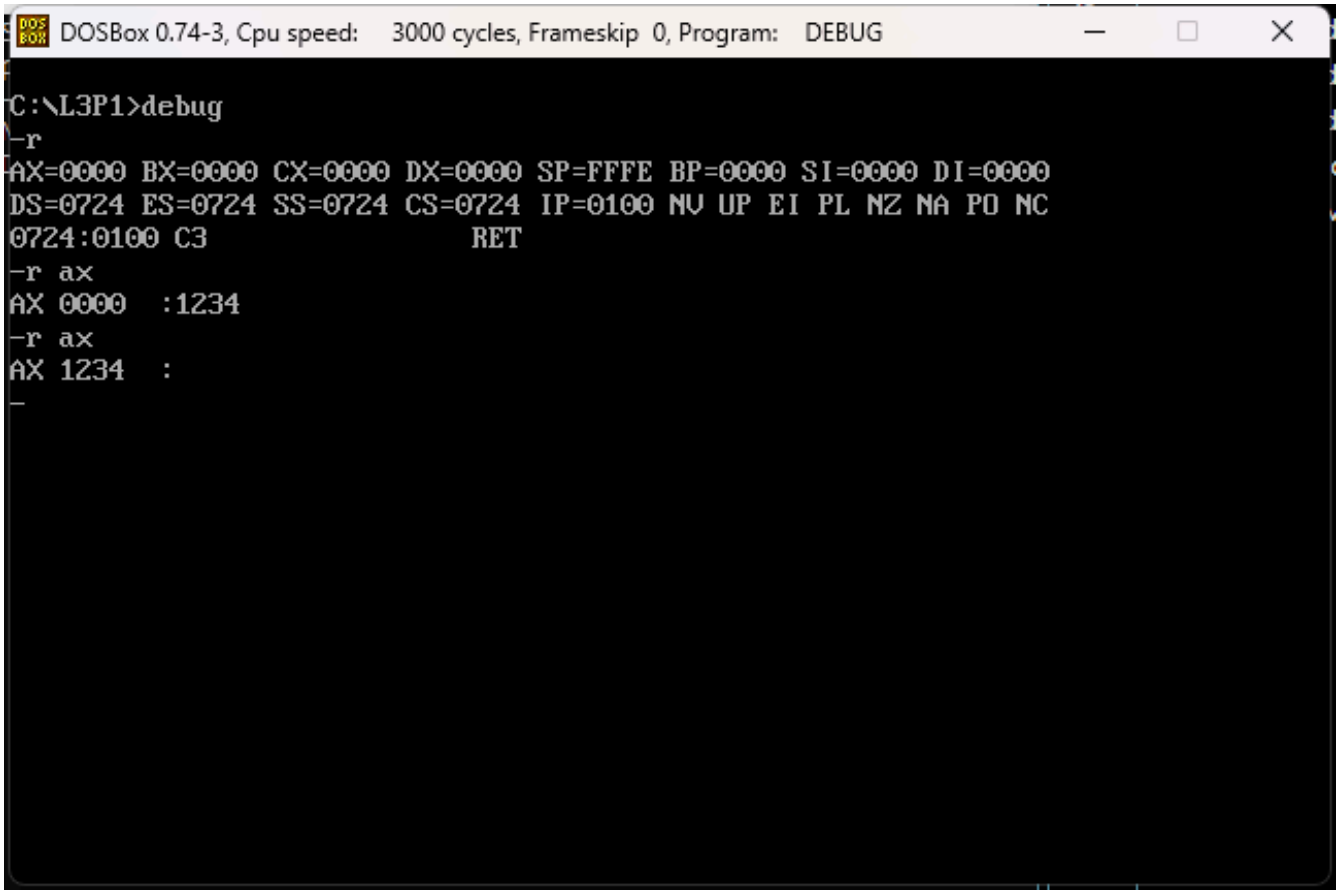
- El volcado completo del programa LOOP requiere 13 bytes (`0100`–`010C`), no 12; el ensamblador observado eligió `83 C0 02` para `ADD AX,0002`.
- Las capturas reales documentan los comandos y resultados; los bucles largos se dividieron en varias imágenes para mantener legibles los estados de cada instrucción.

Entrega en GitHub

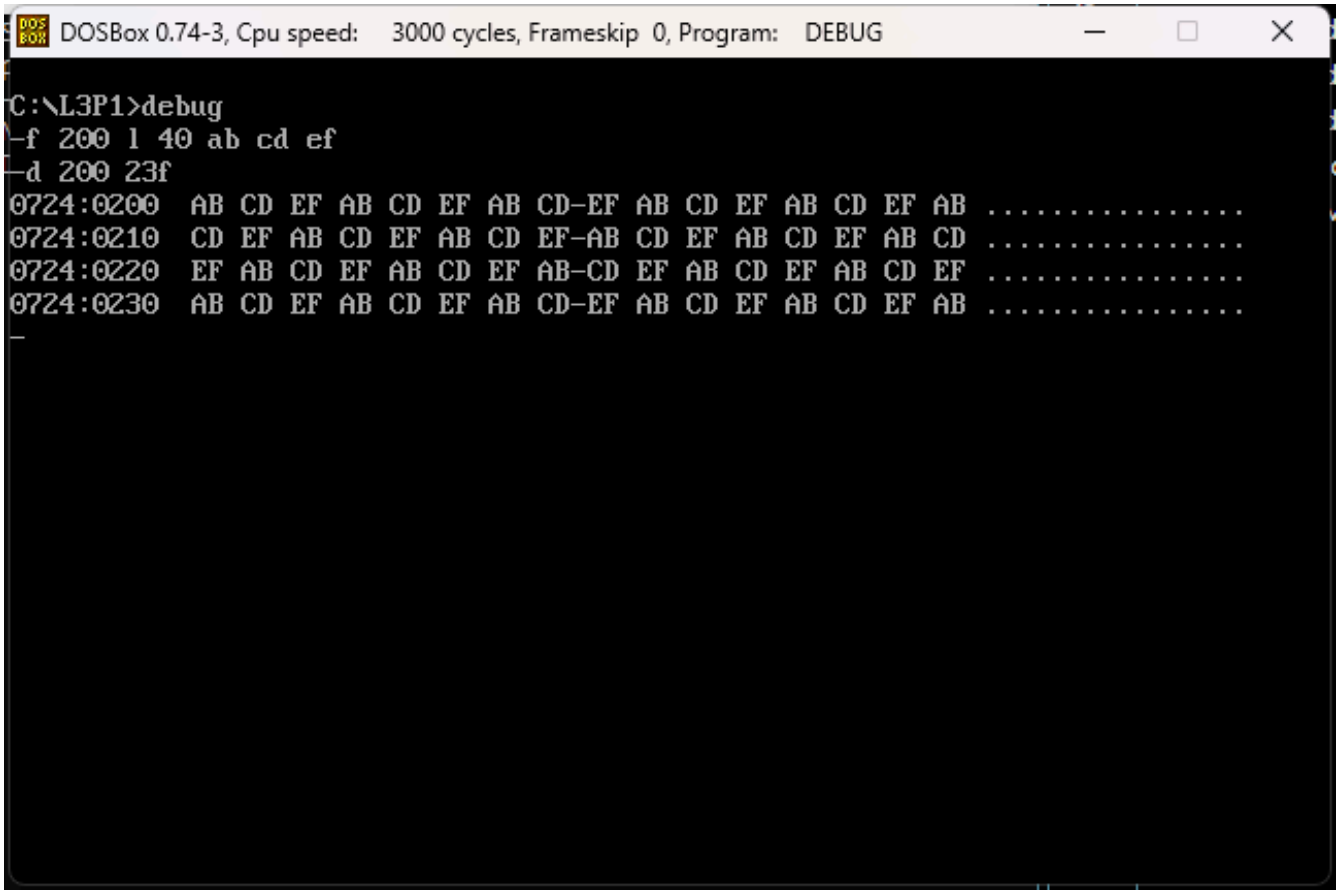
La guía académica solicita un repositorio llamado `apellido-post1-u3`, cuatro checkpoints por parte y seis commits como evidencia del proceso. Esta actividad está organizada dentro del repositorio de curso TrabajosArqui; por tanto, no se afirma que exista un repositorio independiente con aquel nombre ni se crean commits ficticios. El README y las capturas reales están preparados localmente; el historial y la publicación en el destino de GitHub que se comparta con el docente aún deben realizarse.

Anexo ? Evidencias de DOSBox

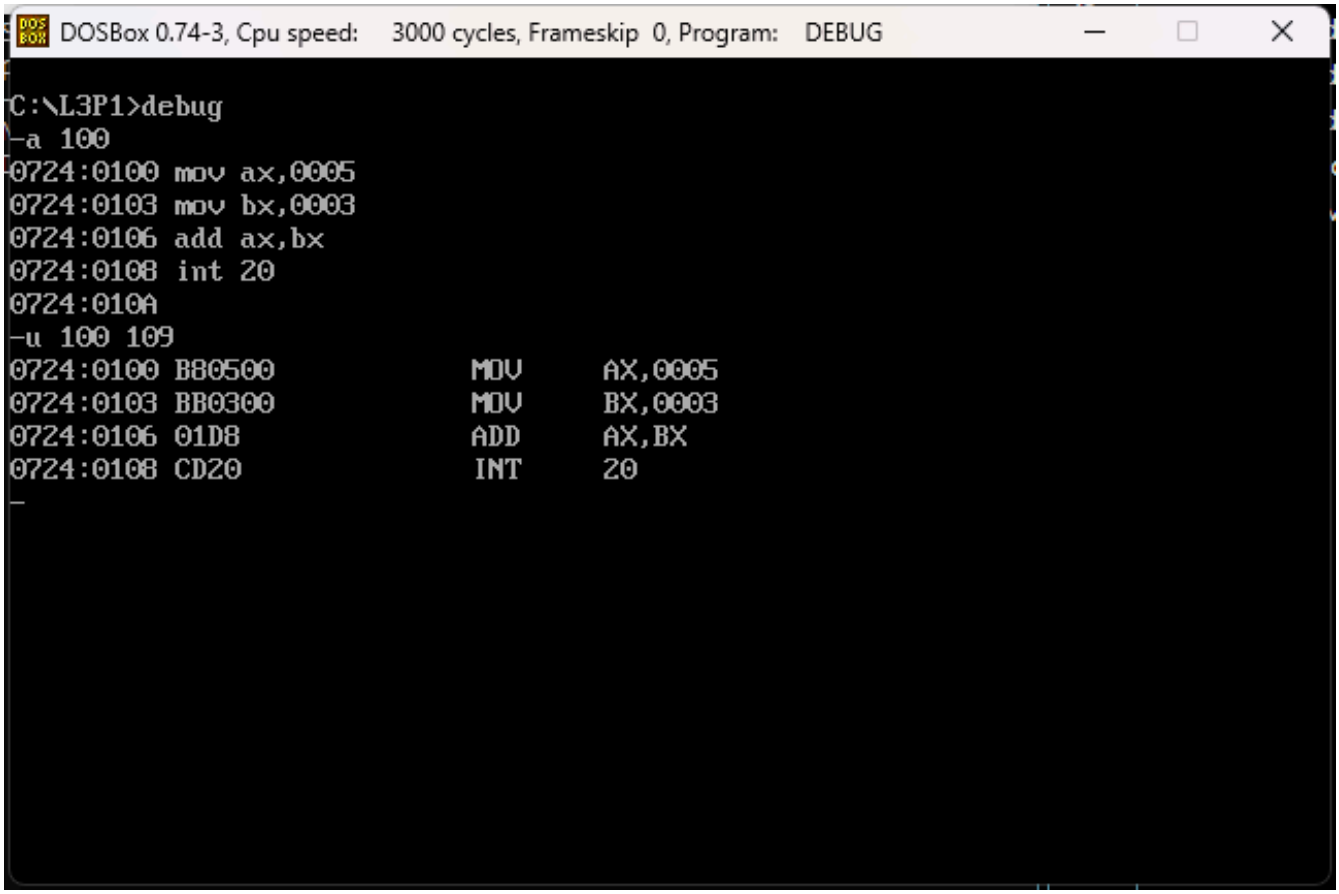
Capturas reales de la ejecuci?n de FreeDOS DEBUG en DOSBox 0.74-3. Segmento observado: 0724.



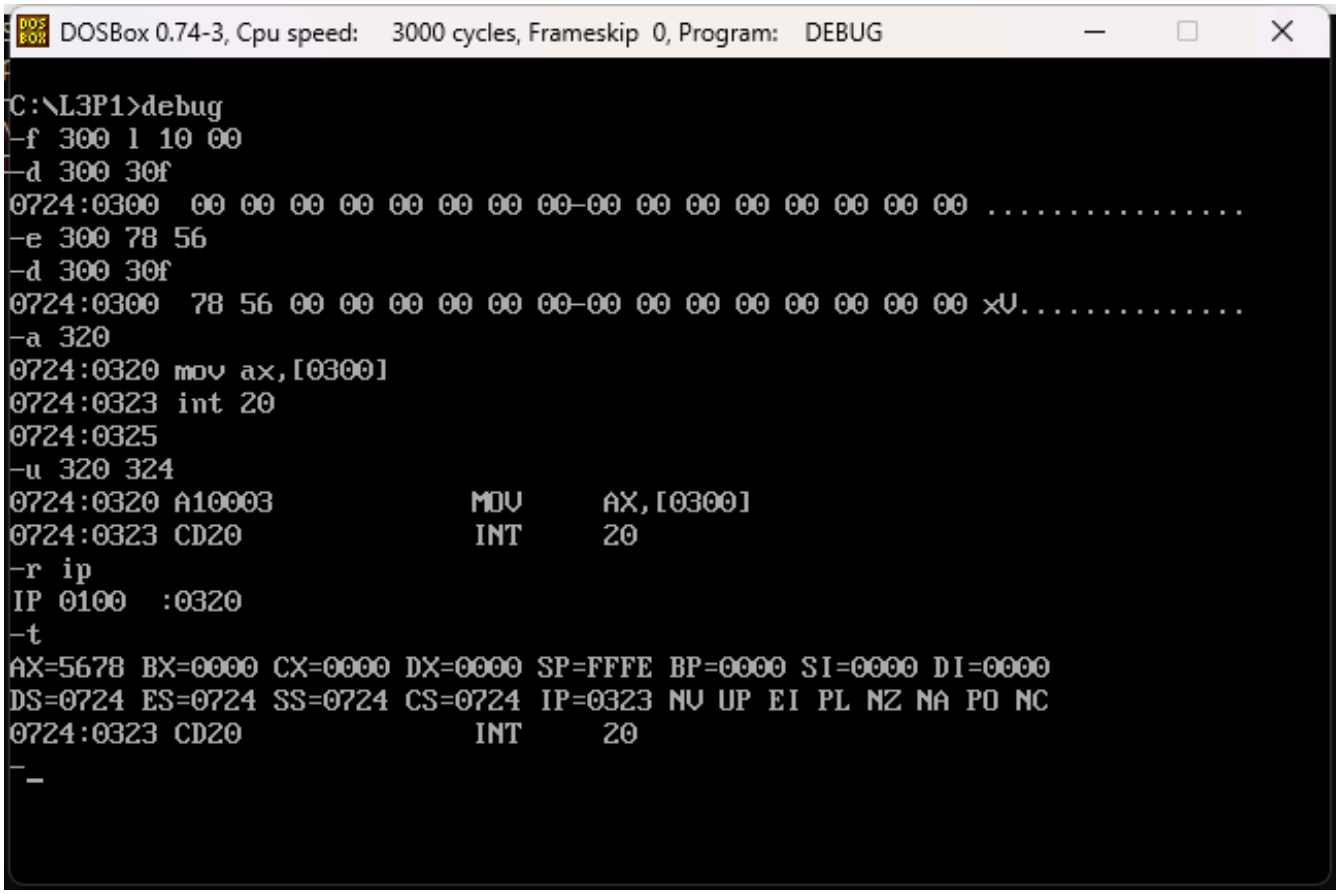
CP1_registros.png



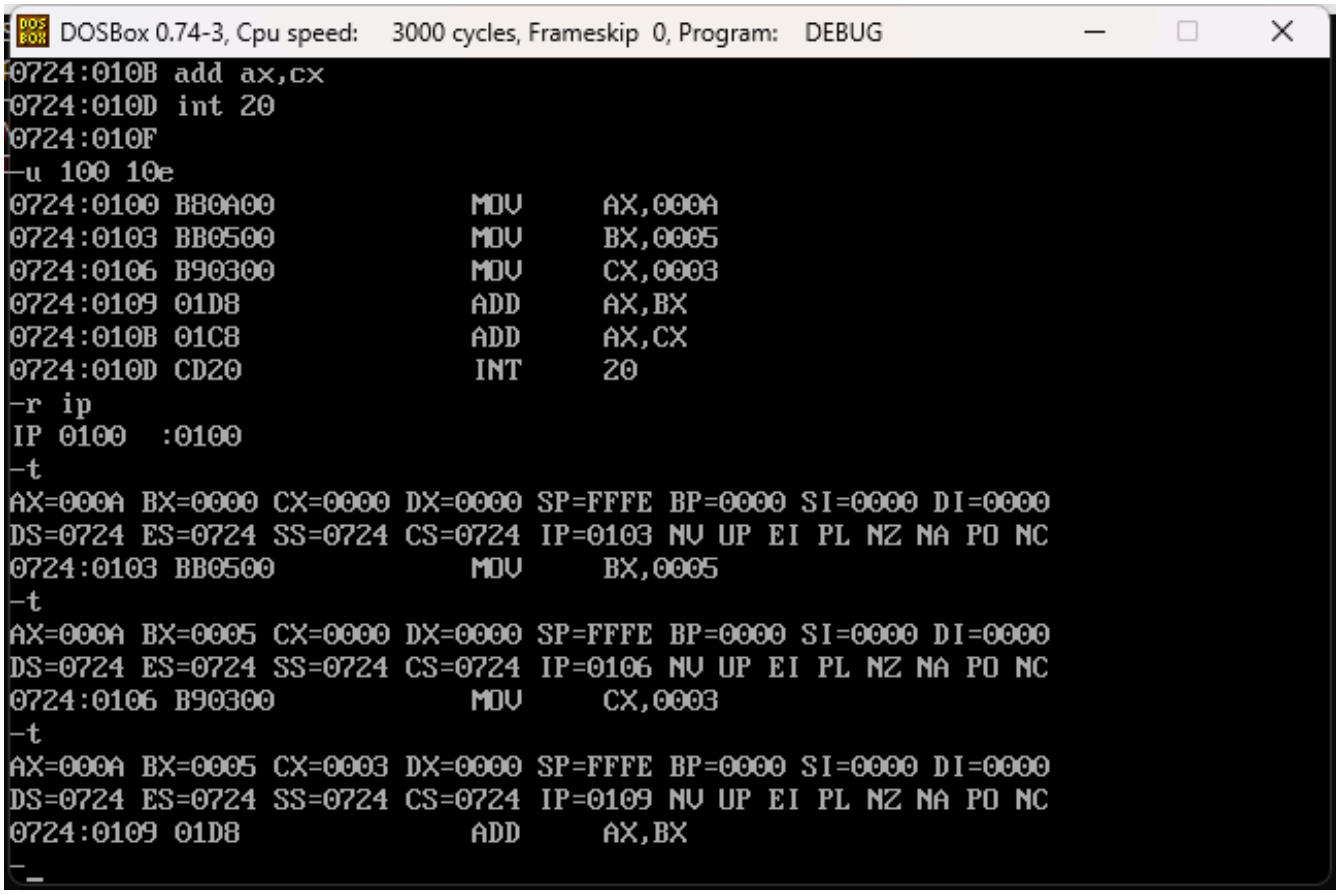
CP2_volcado_memoria.png



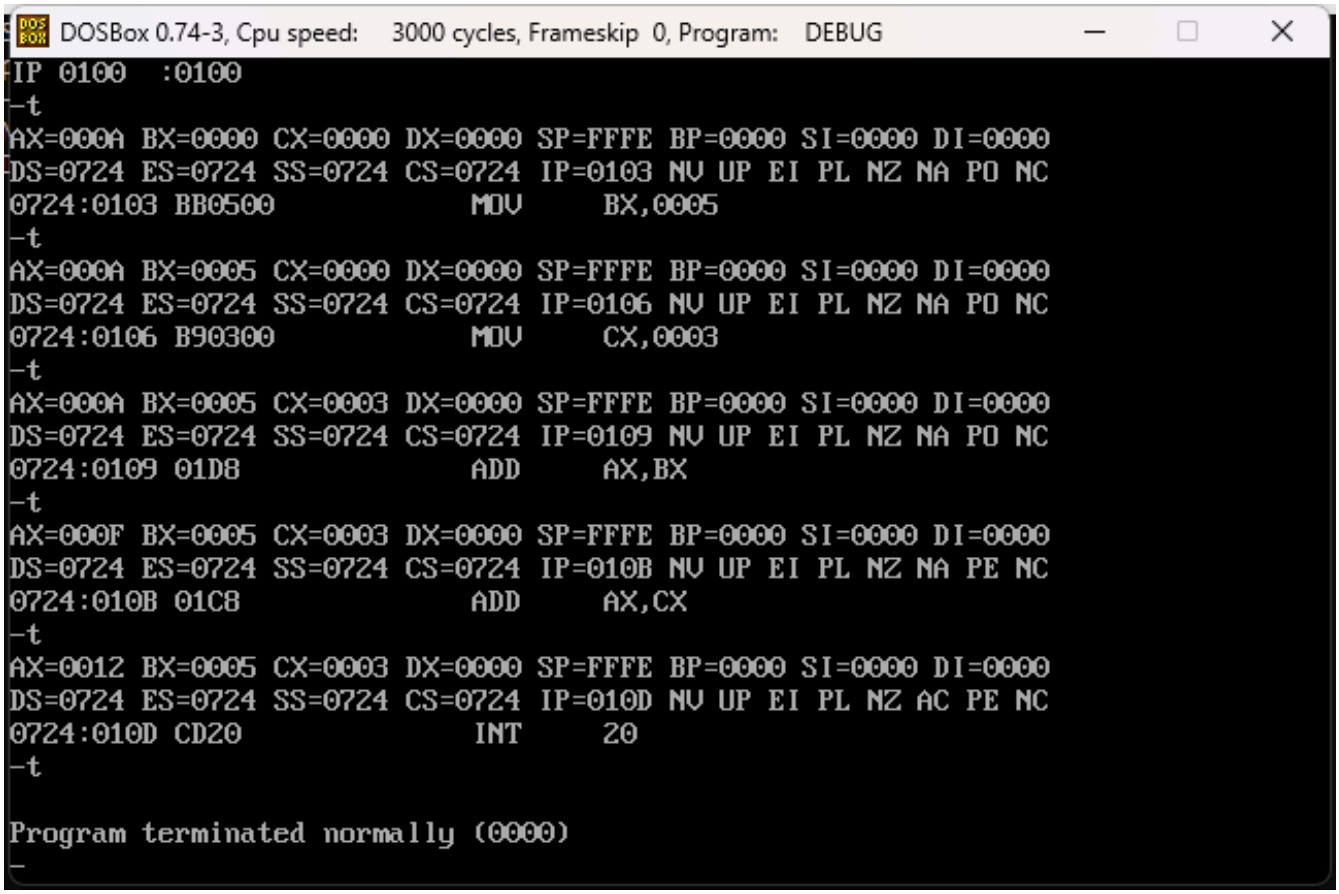
CP3_ensamblado_desensamblado.png



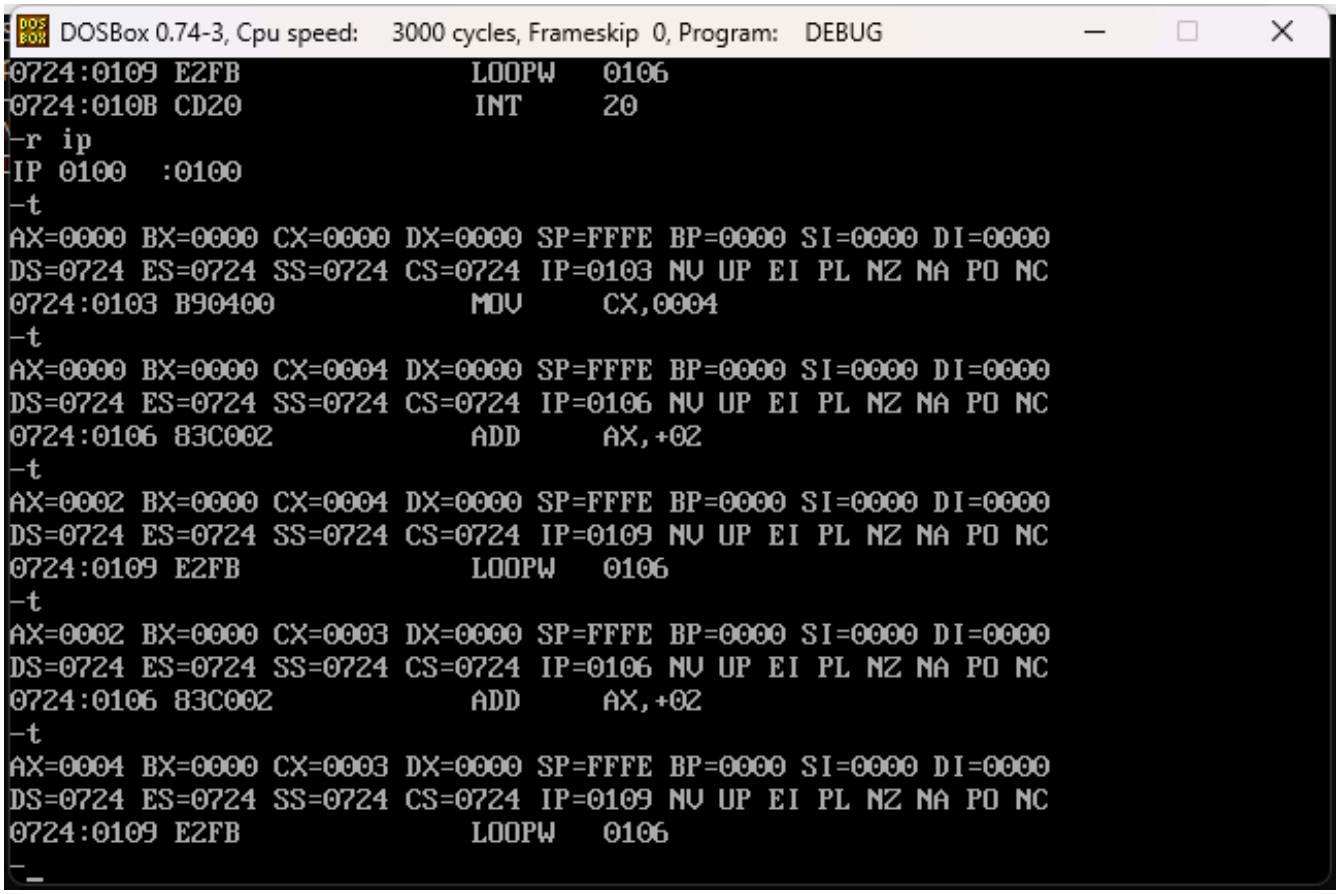
CP4_memoria_direccionamiento.png



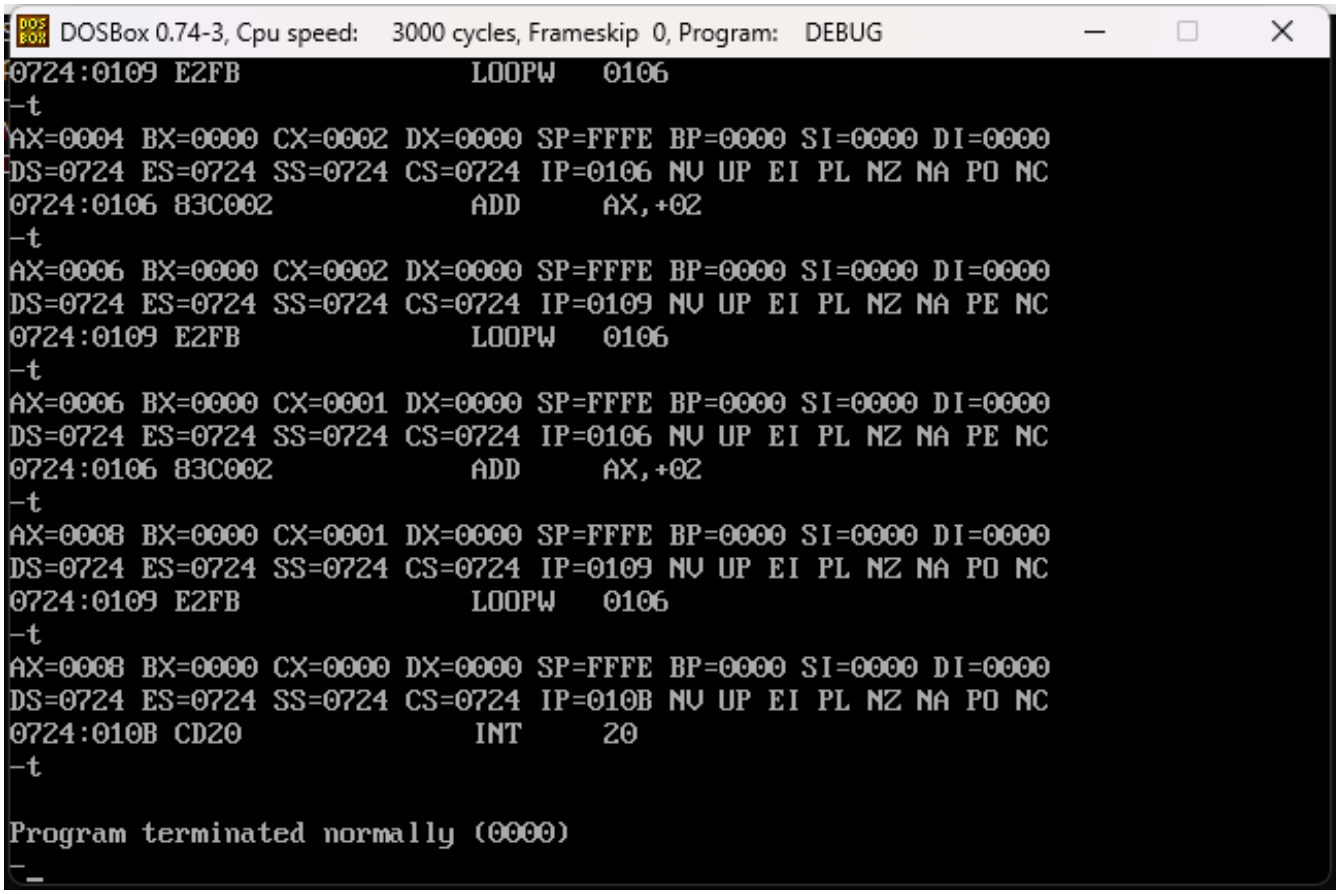
CP1_traza_suma_inicio.png



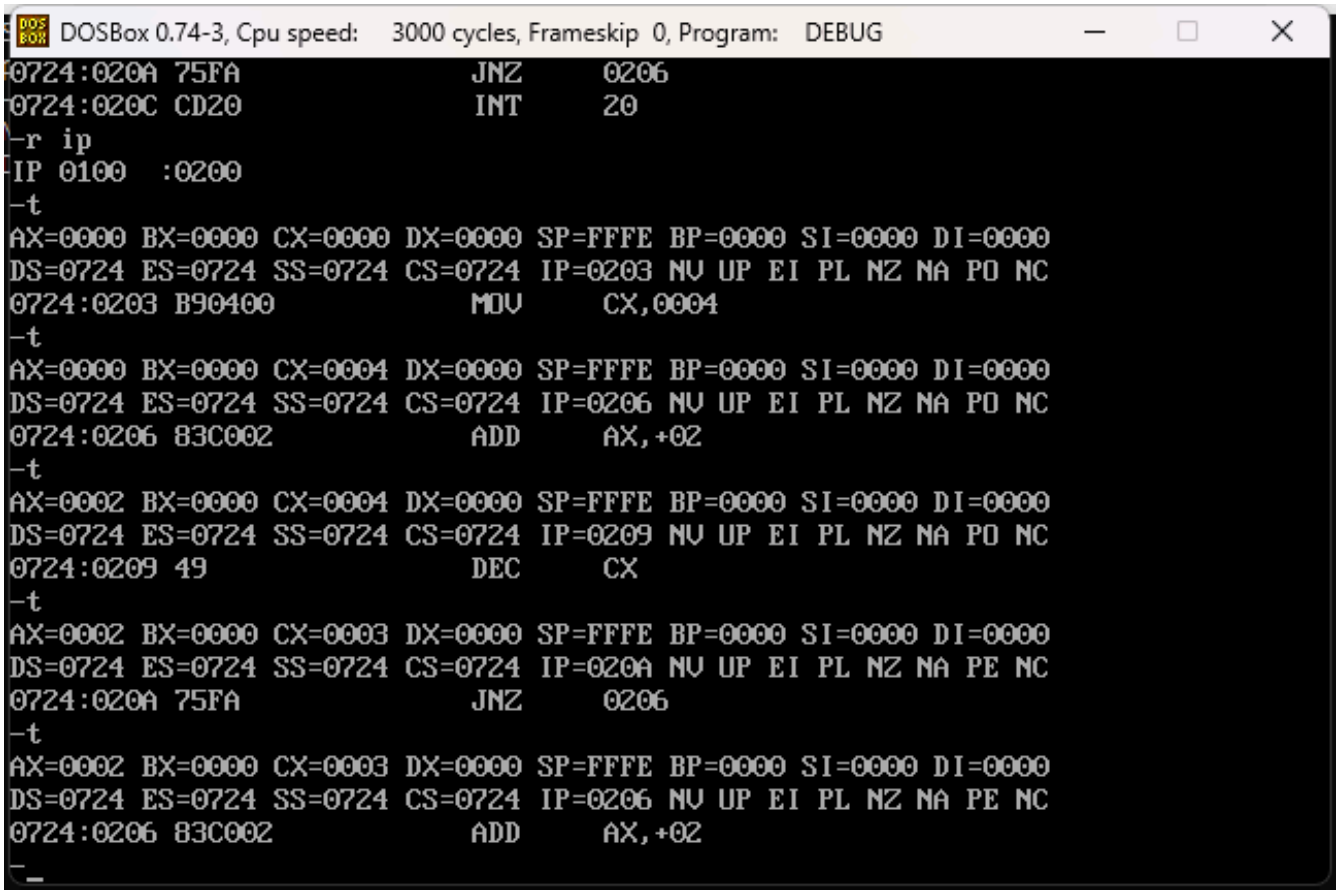
CP1_traza_suma.png



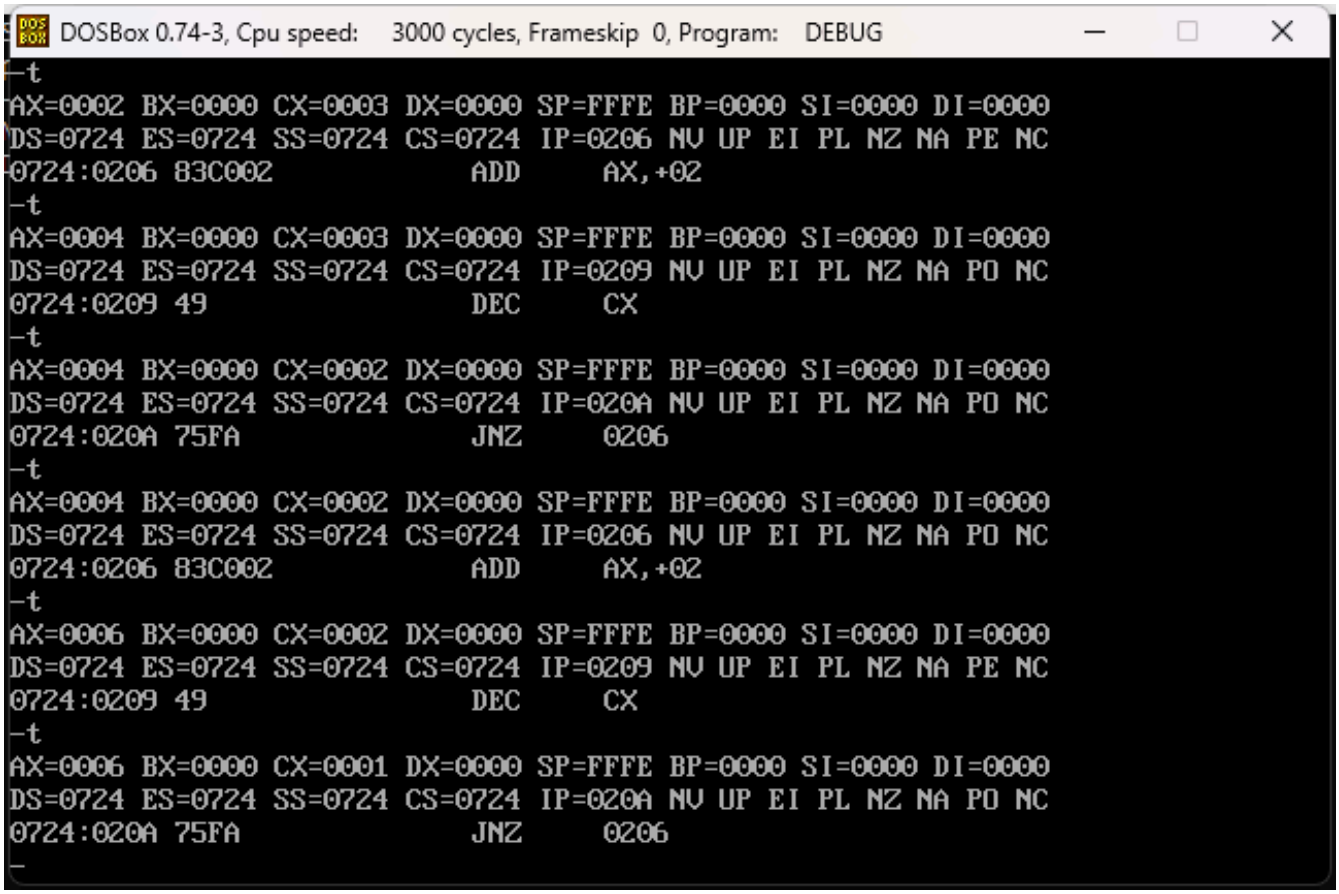
CP2_traza_loop_inicio.png



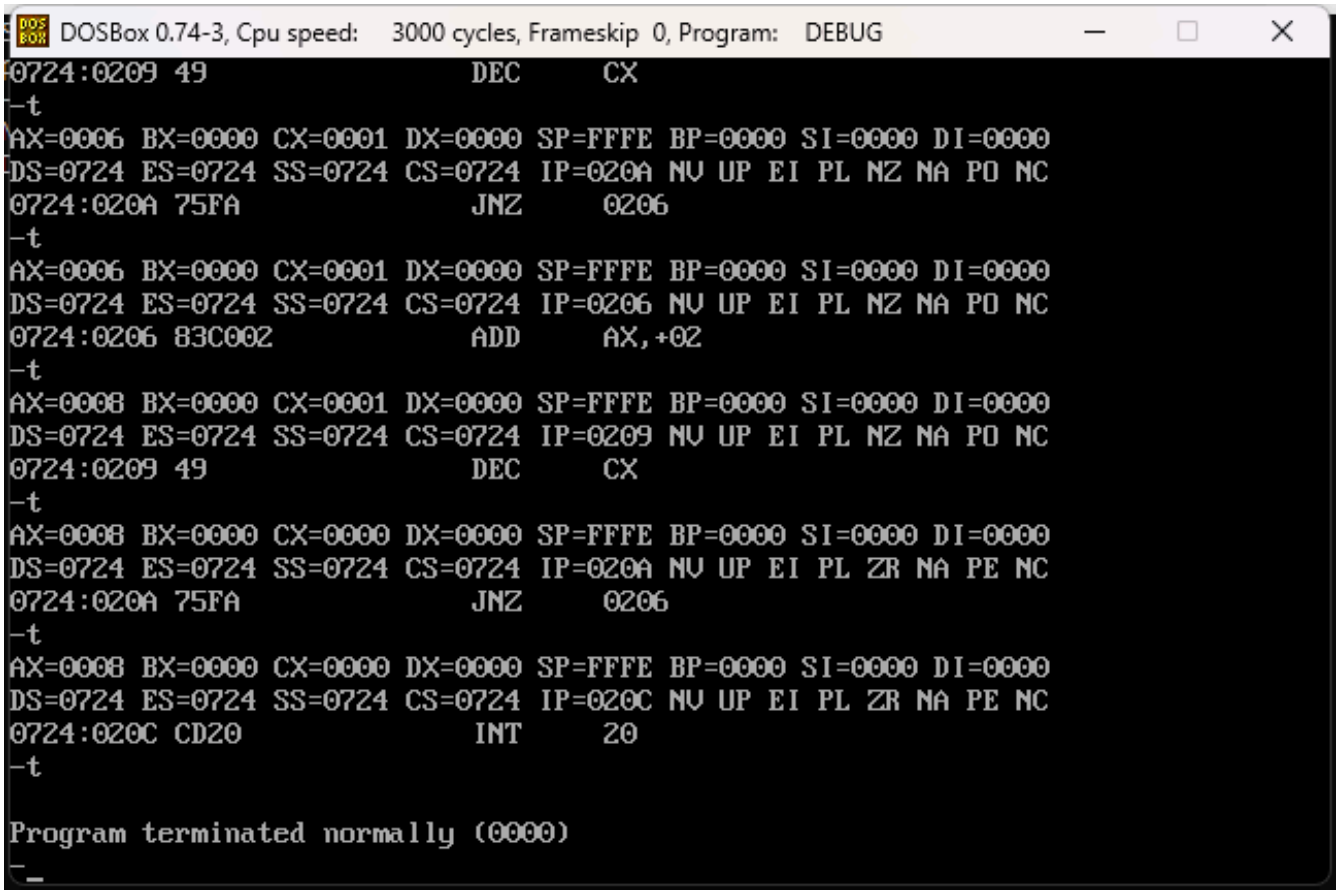
CP2_traza_loop.png



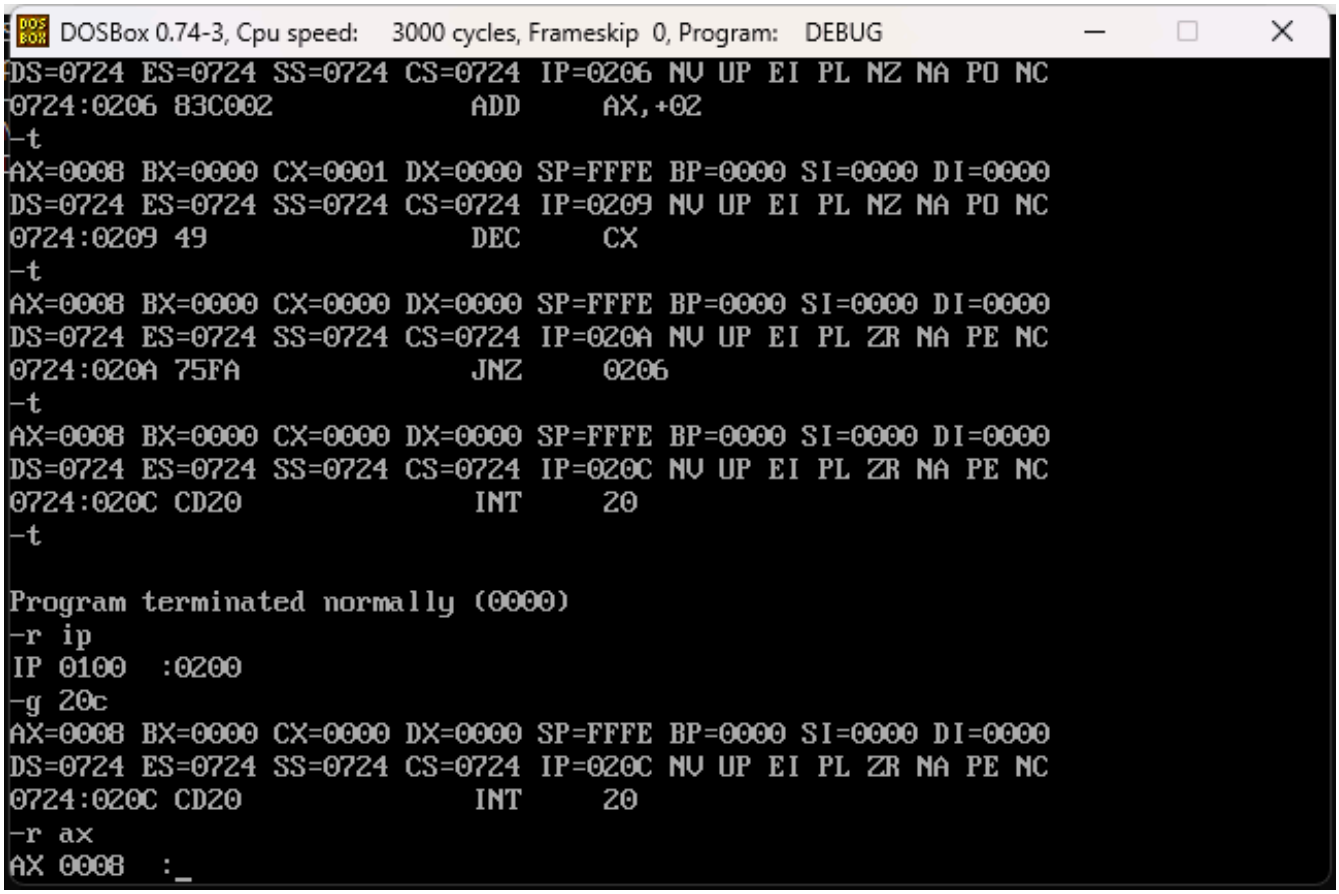
CP3_traza_dec_jnz_inicio.png



CP3_traza_dec_jnz_medio.png



CP3_traza_dec_jnz.png



CP3_verificacion_G.png